■ NEXUS LUXURY E-COMMERCE

Full-Stack Interview Handbook & Q&A; Master Guide

Comprehensive preparation manual for Full-Stack, Frontend, Backend, and System Design interviews based on the production NEXUS platform.

Stack: Next.js 15, React 19, ASP.NET Core 9, PostgreSQL	**Architecture:** Headless, Clean Architecture, REST API
Candidate Role: Full-Stack / Senior Web Engineer	**Deployment:** Vercel (Edge CDN) + Render (Docker)

1. Project Overview & Elevator Pitches

30-Second Elevator Pitch (For Introductions):

"NEXUS is an enterprise-grade luxury fashion e-commerce platform built with Next.js 15 App Router, ASP.NET Core 9 Web API, and Neon Serverless PostgreSQL. It combines a haute horlogerie jewel-tone design system with high-converting e-commerce patterns inspired by AJIO and SHEIN—including a multi-level mega-menu, an interactive promotional coupon engine, a resilient Cash on Delivery checkout pipeline with automated address fallbacks, and a real-time admin portal with dynamic order status glow effects."

2-Minute Architectural Walkthrough (When Interviewers ask 'Walk me through your architecture'):

- **Frontend (Next.js 15 & React 19):** Strategic mix of Server Components for instant SSR & SEO, and Client Components for interactive UI (Zustand for lightweight state, Radix UI primitives, Lucide icons, and Tailwind CSS). Shared JS bundle is only 103 kB.
- **Backend (.NET 9 C#):** Built on Clean Architecture (EShop.Core, EShop.Infrastructure, EShop.API) using ASP.NET Core Kestrel. Features custom JSON deserializers, IP rate limiting, and centralized RFC 7807 ProblemDetails middleware.
- **Persistence (Neon Serverless PostgreSQL):** Managed via Entity Framework Core 9 with code-first migrations and safe, idempotent startup seeding. ACID transactions guarantee zero inventory overselling.
- **DevOps & Hosting:** Dockerized with multi-stage builds. Frontend deployed on Vercel Edge Network; backend deployed as a containerized web service on Render with zero-trust CORS and automated health probes.

2. Architectural Justifications: Why X Over Y?

Next.js 15 App Router vs. Traditional SPA (Vite / CRA)

Why Next.js: E-commerce catalog discoverability relies entirely on search engine indexing (SEO). Traditional SPAs serve a blank HTML shell that harms crawler ranking and First Contentful Paint. Next.js 15 Server Components prerender HTML on the edge while keeping client bundle footprint minimal.

Zustand vs. Redux Toolkit

Why Zustand: Redux introduces excessive boilerplate (actions, reducers, dispatchers) and adds ~15 kB to the bundle. Zustand is under 1.5 kB, has zero boilerplate (hook-based stores), and seamlessly supports local storage persistence with 1-line middleware for cart and wishlist.

ASP.NET Core 9 vs. Node.js / Express

Why .NET 9: In enterprise commerce, type safety and throughput are paramount. Kestrel delivers 5-10x the throughput of single-threaded Node.js event loops, native Dependency Injection eliminates third-party container bloat, and C# strong types eliminate runtime type mismatches.

PostgreSQL (Neon Serverless) vs. MongoDB

Why PostgreSQL: Financial transactions, coupons, and orders demand strict ACID consistency. Relational foreign key constraints prevent orphaned order items, while Neon provides serverless compute scaling and PgBouncer connection pooling.

3. Real-World Engineering Challenges (STAR Method)

Interviewers love hearing how you debug and solve real, messy bugs. Use these stories:

STORY 1: Resolving the 400 Bad Request on Cash on Delivery Orders

Situation: During final checkout testing, placing a Cash on Delivery order threw an ambiguous 400 Bad Request error before reaching business logic.

Task: Track down the root cause in the ASP.NET Core request pipeline and restore seamless order placement.

Action:
1. Audited Serilog logs and discovered the failure stemmed from Dependency Injection: OrdersController injected IHttpClientFactory for Razorpay, but AddHttpClient() was missing in Program.cs. ASP.NET Core threw an InvalidOperationException, which our GlobalExceptionMiddleware converted to a 400 Bad Request.
2. Also detected that the frontend sent addressId: "" (empty string) and payment method "cod" (lowercase string). Default .NET System.Text.Json failed to deserialize empty strings to Guid?.
3. Created custom converters (NullableGuidConverter & PaymentMethodConverter), registered AddHttpClient(), and added a 4-tier fallback address resolver.

Result: Order placement achieved 100% success rate with zero deserialization errors and instant ■0 upfront COD confirmation.

STORY 2: Windows Background Process DLL Locking (MSB3026)

Situation: Rapid hot-reloading on Windows caused builds to fail with MSB3026: Could not copy EShop.Infrastructure.dll because it is being used by another process: EShop.API.

Task: Safely terminate locked binaries without corrupting database transactions or halting development.

Action: Identified the lingering background worker holding file handles via PowerShell Get-Process, cleanly terminated orphaned PIDs, and configured production release publishing (dotnet publish -c Release -o backend/publish) to isolated directories with .gitignore rules.

Result: Completely eliminated binary lock collisions and separated live runtime executions from release builds.

4. System Design & Scalability Scenarios

Q: How would you handle 100,000 concurrent users during a Black Friday flash sale?

1. Edge Caching (CDN): Cache static assets, product images, and category landing pages on Vercel Edge / Cloudflare. The origin API receives zero traffic for read-only pages.
2. Redis Distributed Cache: Place Redis in front of PostgreSQL for /api/products and /api/categories. Catalog queries respond from memory in < 2ms.
3. Atomic Stock Decrement: Prevent inventory race conditions with SQL atomic decrements:
UPDATE Products SET Stock = Stock - @qty WHERE Id = @id AND Stock >= @qty;
If rows affected is 0, the item sold out concurrently; abort cleanly without table locking.
4. Async Order Pipeline: Decouple checkout from fulfillment via message queues (RabbitMQ / Kafka). Return 202 Accepted with an order ID in 40ms, while background workers handle notifications and warehouse dispatch.

5. Top 25 Technical Interview Questions & Model Answers

Q1: What is the difference between Server and Client Components in your app?

Server Components execute exclusively on the server, generating lightweight HTML with 0 kB JavaScript bundle impact. Client Components ('use client') execute in the browser and are reserved for interactive features like CartDrawer, AuthModal, search debounce, and interactive filters.

Q2: How does your shopping cart persist across browser refreshes?

Zustand's persist middleware automatically syncs the cart state to localStorage. When a user authenticates, the client synchronizes the local cart with the server via /api/cart, ensuring cart continuity across devices.

Q3: How do you prevent layout shifts (CLS) when loading luxury product images?

We use Next.js <Image /> with explicit aspect ratios (aspect-[3/4]), placeholder shimmer skeleton cards, and CSS overflow containers so the layout footprint is reserved before the image bytes download.

Q4: How does error handling work across the frontend?

src/lib/api.ts implements a centralized transformError interceptor that unwraps ASP.NET Core RFC 7807 ProblemDetails or validation arrays into clean toast messages for the user.

Q5: What is the luxury color harmony system you implemented?

Built on Obsidian Black (#0A0A0A) and Imperial Gold (#C9A84C), elevated with haute horlogerie jewel tones: Emerald Green (Delivered / In-Stock), Ruby Red (Cancelled / Out of Stock / Sale), Sapphire Blue (Shipped / In-Transit), and Topaz Amber (Pending / COD).

Q6: What is Clean Architecture and why did you use it?

It enforces separation of concerns: Core has zero dependencies and holds business entities and interfaces; Infrastructure implements database queries and external services; API handles HTTP presentation. This allows swapping databases or adding background workers without touching core domain rules.

Q7: What is the Unit of Work pattern?

IUnitOfWork aggregates multiple repository calls into a single database context transaction. Calling SaveChangesAsync() executes atomically across all modified entities, guaranteeing all-or-nothing data consistency.

Q8: Why did you build custom JSON Converters?

Standard System.Text.Json throws 400 Bad Request if an empty string is posted to a Guid? or if an enum string casing doesn't match exactly. Our custom converters normalize these inputs, making the API resilient against real-world client quirks.

Q9: How does your rate limiting middleware work?

An IP-based rate limiter backed by IMemoryCache tracks request counts per IP. General endpoints allow 100 requests/minute, while sensitive auth endpoints (/api/auth) are throttled to 5 requests/minute to mitigate brute-force attacks.

Q10: How does your Global Exception Middleware handle errors?

It wraps the HTTP request in a try-catch block, logs stack traces via Serilog, and maps exceptions to standardized RFC 7807 ProblemDetails responses (400 for validation, 401 for unauthorized, 404 for not found, 500 for unhandled) without exposing internal server secrets.

Q11: How do you prevent SQL Injection?

Entity Framework Core uses parameterized SQL queries exclusively. User input is treated strictly as data parameters rather than executable SQL code.

Q12: How are user passwords secured?

We use BCrypt.Net-Next with automated salt generation. Passwords are never stored in plaintext and cannot be decrypted through rainbow tables.

Q13: How does your JWT token rotation work?

Short-lived access tokens (15 minutes) are paired with long-lived refresh tokens (7 days). On access token expiry, Axios silently calls /api/auth/refresh, receives a fresh access token, and retries the original request without user interruption.

Q14: How do you handle database migrations in production?

During startup, Program.cs executes await db.Database.MigrateAsync(), ensuring newly added migration classes apply cleanly without manual SQL scripts.

Q15: What is idempotent database seeding?

DbInitializer.SeedAsync() checks for record existence before inserting. Server restarts can re-run the seeder repeatedly without creating duplicate categories, products, or admin accounts.

Q16: How do you prevent coupon tampering?

The client never decides the discount amount. The backend re-validates the coupon against the database at the instant of order placement, checking expiry, usage count, minimum order value, and percentage caps.

Q17: Why did you prioritize Cash on Delivery (COD)?

In many high-growth fashion markets, Cash on Delivery accounts for 50-70% of e-commerce checkouts. Supporting reliable COD with ■0 upfront payment dramatically reduces cart abandonment.

Q18: How do you generate unique, readable order numbers?

OrderNumberService generates deterministic strings formatted as ORD-YYYYMMDD-XXXXXX, combining the date with a unique uppercase GUID slice.

Q19: How does the 4-tier address fallback resolver work?

If an order lacks a saved address ID, it falls back to: 1) Inline shipping address, 2) User default saved address, 3) User account profile, preventing order rejections caused by missing address IDs.

Q20: How does the Admin Order Status workflow operate?

Admins can transition orders through Confirmed → Shipped → Delivered or trigger Cancelled. Status changes trigger live badge updates across the admin table and customer order tracking timelines.

Q21: What was your most valuable technical takeaway from this project?

Understanding that enterprise software quality lives at the boundaries: serialization boundaries (JSON converters), network boundaries (CORS, reverse proxy forwarded headers), and transaction boundaries (atomic inventory decrements).

Q22: How is the application containerized?

A multi-stage Dockerfile compiles and publishes the .NET 9 API using the full SDK, then copies only the lean binaries to an aspnet:9.0 runtime image, keeping image size minimal and secure.

Q23: How do you monitor container health in cloud hosting?

The ASP.NET Core /health endpoint is mapped in Program.cs and probed by Render/Kubernetes orchestrators to automatically restart unresponsive containers.

Q24: How does dynamic CORS support preview deployments?

The CORS policy dynamically inspects the request origin: any host ending with .vercel.app, .netlify.app, or .onrender.com is approved on the fly, allowing seamless preview deployments.

Q25: What is the first enhancement you would build next?

Integrating an asynchronous message broker (RabbitMQ) for background order processing, real-time inventory drop timers with Redis, and transactional email confirmations via Resend.